

Name of the Student	N.Jeevan Sai
Reg. No	192525169
GitHub Link	https://github.com/Nagellajeevansai/MLA0405-Deep-Learning

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL

2

Scenario-Based Questions

COURSE NAME : Deep Learning
Faculty : Dr. Arul Raj A.M

COURSE CODE : MLA0405
Slot : D

COURSE OUTCOME COVERED

CO 2:Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

Duration: 90 minutes

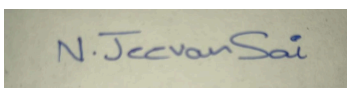
Marks: 25

Code of Conduct:

I (N.Jeevan sai / Reg No:192) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Scenario-Based Questions

CO-AT2

QUESTIONS: 05

Total Marks:25

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Solution:

An autoencoder learns the underlying structure of ECG signals without requiring labeled data. It is designed to reconstruct its input as accurately as possible. When trained on mostly normal ECG patterns, the network learns the characteristics of healthy heart signals. During testing, abnormal ECG patterns usually produce a larger reconstruction error because the model cannot reconstruct signals that differ significantly from the learned normal patterns.

Autoencoder architecture

An autoencoder consists of two main parts:

- Encoder: Compresses the ECG signal into a low-dimensional representation.
- Decoder: Reconstructs the original ECG signal from the compressed representation.

Workflow

ECG anomaly detection workflow

ECG records

Preprocessing and normalization

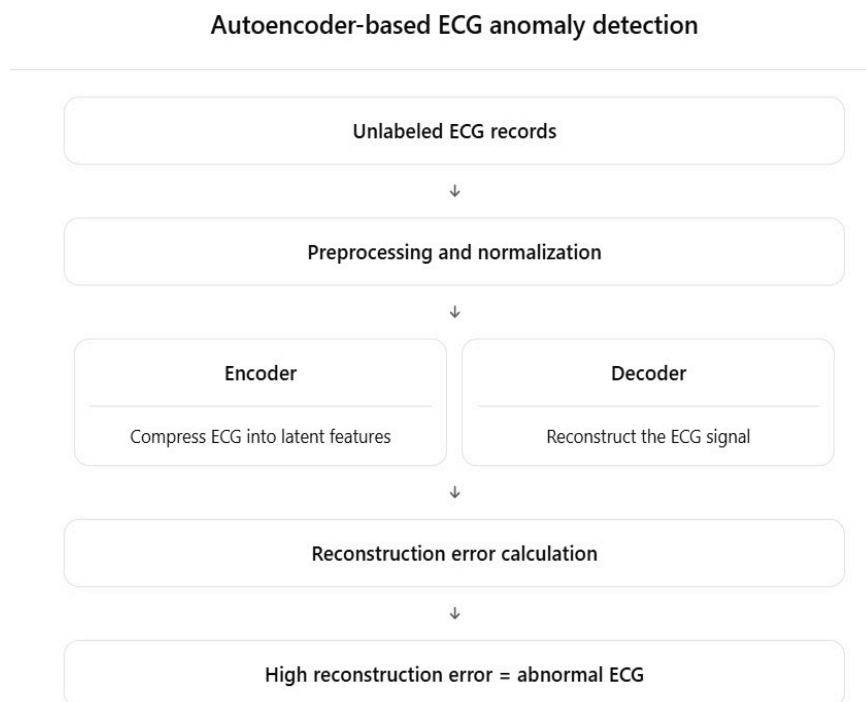
Train autoencoder

Learn compressed features

Reconstruct ECG

Compute reconstruction error

Abnormal ECG detection



Detailed steps

1. Collect ECG records from hospital monitoring systems.
2. Remove noise and normalize the signals.
3. Train the autoencoder using the available unlabeled data.
4. The encoder extracts important heartbeat features.
5. The decoder reconstructs the ECG waveform.
6. Calculate the reconstruction error.
7. If the error exceeds a predefined threshold, classify the ECG as abnormal.

Advantages

- Does not require labeled abnormal ECG data.
- Learns meaningful ECG representations automatically.
- Detects rare and unknown abnormalities.
- Can process large volumes of ECG records efficiently.
- Supports real-time monitoring in intensive care units.

Limitations

- Performance depends on the quality of normal training data.
- Threshold selection for anomaly detection is important.
- Extremely noisy ECG signals may affect reconstruction quality.

Conclusion

Autoencoders are highly effective for ECG anomaly detection because they learn normal cardiac patterns and identify abnormal signals through reconstruction error. Their ability to work with unlabeled data makes them an ideal solution for hospitals with large ECG databases.

2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

Solution:

Dimensionality reduction for customer segmentation

A retail company has customer purchase data containing hundreds of features, including products purchased, purchase frequency, spending amount, discounts used, and shopping categories. High-dimensional data makes customer segmentation difficult. An autoencoder can perform nonlinear dimensionality reduction and generate compact customer representations for clustering.

Representation learning

Representation learning automatically learns useful features from raw data. Instead of manually selecting features, the autoencoder learns compressed representations that preserve important purchasing behavior.

Autoencoder structure

Customer representation learning

Purchase data

Encoder

Latent

representation

Decoder

Reconstructed purchase data

Workflow

1. Prepare customer purchase records.
2. Normalize numerical features.
3. Train the autoencoder to reconstruct the original purchase data.
4. Extract the latent representation from the bottleneck layer.
5. Apply clustering algorithms such as K-means or hierarchical clustering.
6. Analyze customer segments.

Example

Suppose the original dataset contains 500 purchase-related features.

The autoencoder compresses them into 20 latent features.

These features may represent:

- premium shoppers,
- frequent buyers,
- discount-oriented customers,
- seasonal shoppers,
- brand-loyal customers.

Advantages

- Reduces dimensionality.
- Removes redundant information.
- Captures nonlinear relationships.
- Improves clustering accuracy.
- Speeds up segmentation algorithms.
- Reduces storage and computational requirements.

Applications

- Personalized product recommendations.
- Targeted marketing campaigns.
- Customer lifetime value analysis.
- Churn prediction.
- Inventory optimization.

Conclusion

Autoencoders provide an efficient dimensionality reduction technique by learning compact customer representations. These representations improve customer segmentation quality and enable better business decision-making.

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

Solution:

Improving facial recognition performance

A facial recognition system trained with a shallow neural network performs poorly because it cannot capture the complex patterns present in facial images. The recommended solution is to increase the network depth, optimize the network width, and use suitable activation functions.

Why shallow networks fail

A shallow network learns only simple patterns such as edges and brightness variations. Facial recognition requires learning:

- facial contours,
- eye shapes,
- nose structure,
- mouth position,
- facial symmetry,
- expressions,
- lighting variations.

These complex features require multiple hidden layers.

Increase network depth

Use a deep neural network with several hidden layers. Example:

Input $\rightarrow$ H1 $\rightarrow$ H2 $\rightarrow$ H3 $\rightarrow$ H4 $\rightarrow$ H5 $\rightarrow$ Output

Each layer learns increasingly complex features.

Hierarchical feature learning

Layer	Features learned
--------------	-------------------------

Layer 1 Edges and textures

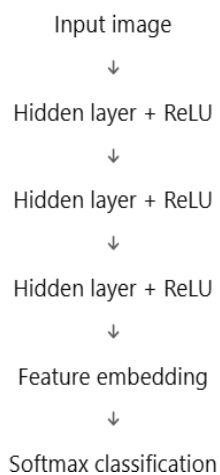
Layer 2 Eyes and nose

Layer 3 Facial components

Layer 4 Complete face representation

Layer 5 Identity-specific features

Deep facial recognition network



Increase network width

Use more neurons per hidden layer to improve the model's capacity.

Activation functions

Use ReLU in hidden layers.

$$f(x) = \max(0, x)$$

Advantages:

- faster convergence,
- reduced vanishing gradient problem,
- sparse activations,
- better deep network training.

Use Softmax in the output layer for multi-class facial recognition.

Improved architecture

Deep facial recognition network

Input image

Hidden layer + ReLU

Hidden layer + ReLU

Hidden layer + ReLU

Feature embedding

Softmax

classification

Additional improvements

- Batch normalization
- Dropout regularization
- Residual connections
- Data augmentation
- Learning rate scheduling

Conclusion

Increasing network depth, appropriately increasing width, and using ReLU activation functions significantly improves facial recognition performance by enabling hierarchical feature learning and more effective optimization.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

Solution:

Movie recommendation using RBMs

A streaming service wants to recommend movies based on users' viewing history without labeled preference data. A Restricted Boltzmann Machine (RBM) is an effective unsupervised learning model for collaborative filtering.

RBM architecture

An RBM contains:

- Visible layer: user-movie interactions.
- Hidden layer: latent preference features.

RBM recommendation model

Visible layer

Movie A

Movie B

Movie C

Movie D

Hidden

layer

Action preference

Comedy preference

Romance preference

Working principle

Suppose a user watches:

- action movies,
- science fiction,
- adventure films.

The RBM learns hidden preferences such as:

- action preference,
- thriller preference,
- fantasy preference.

Training process

1. Create a user-movie interaction matrix.
2. Initialize weights randomly.
3. Perform forward propagation.
4. Reconstruct visible units.
5. Update weights using Contrastive Divergence.

6. Repeat until convergence.

Recommendation generation

The trained RBM predicts missing movie

preferences. Example:

Observed:

- Movie A = watched
- Movie B = watched
- Movie C = not watched

Predicted probability:

- Movie C = 0.92

Recommend Movie C.

Advantages

- Works with unlabeled data.
- Learns hidden user preferences.
- Handles sparse rating matrices.
- Captures nonlinear user-movie relationships.
- Improves personalized recommendations.

Applications

- Movie recommendation
- Music recommendation
- Video streaming
- Online shopping
- Content personalization

Conclusion

RBM's learn latent user preferences from viewing history and predict unseen movie interests, making them an effective unsupervised recommendation technique.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

Solution:

Defective product detection from sensor readings

A manufacturing company collects sensor readings every second from machines and wants to detect defective products automatically. A deep learning solution using representation learning and autoencoder-based neural networks can effectively identify defects.

Problem characteristics

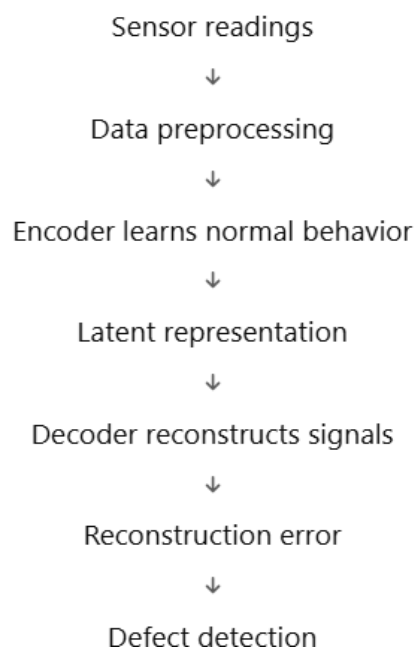
Sensor data includes:

- temperature,
- pressure,
- vibration,
- motor current,
- rotation speed,
- humidity.

Most products are normal, while defective products are rare.

Proposed architecture

Manufacturing defect detection pipeline



Use a deep autoencoder for anomaly detection.

Manufacturing defect detection pipeline

Sensor readings

Data preprocessing

Encoder learns normal behavior

Latent representation

Decoder reconstructs signals

Reconstruction error

Defect detection

Workflow

1. Collect continuous sensor data.
2. Clean and normalize the readings.
3. Train the autoencoder on normal production data.
4. Learn compressed representations of healthy machine behavior.
5. Reconstruct incoming sensor data.
6. Calculate reconstruction error.
7. Trigger an alert when the error exceeds a threshold.

Representation learning

The encoder automatically discovers important features such as:

- vibration signatures,
- thermal behavior,
- pressure correlations,
- equipment operating states,
- wear and tear indicators.

Neural network components

- Multiple hidden layers
- ReLU activation
- Bottleneck representation
- Mean Squared Error (MSE) loss
- Adam optimizer

Benefits

- Real-time defect detection.
- Early identification of machine faults.
- Reduced production waste.
- Lower maintenance costs.
- Improved product quality.

- Predictive maintenance support.

Practical applications

- Automobile manufacturing
- Semiconductor fabrication
- Pharmaceutical production
- Textile manufacturing
- Food processing

Conclusion

A deep autoencoder with representation learning is an effective manufacturing defect detection system because it learns normal sensor behavior automatically and identifies abnormal products through reconstruction error, enabling accurate and real-time quality control.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence

Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand
---------	-----	--	---	---	---